Results Report of the End-to-End Test Automation Suite: SauceDemo with Playwright and TypeScript

Dangelly Angelica Blandon Negrete

Nuaav — Recruitment Process

Position: QA Automation Engineer

Offline Technical Exercise

June 23, 2026

Abstract

This report documents the results of executing an end-to-end (E2E) automated test suite developed in TypeScript on the Playwright framework, targeting the SauceDemo practice application. The suite verifies the application's five functional modules —authentication, product catalogue, shopping cart, checkout, and logout— and runs across two browser engines (Chromium and Firefox) under a fully parallel execution scheme with session-state reuse. Of a total of 37 tests executed, 36 completed successfully and 1 failed, yielding a pass rate of 97.3%. The root-cause analysis determined that the sole failure stems neither from an application defect nor from an error in the test logic, but rather from a non-deterministic timeout associated with Firefox browser resource contention during parallel execution in the local environment. It is concluded that the application's critical flows are verified, and recommendations are offered to mitigate the observed flakiness.

Keywords: test automation, Playwright, TypeScript, E2E testing, SauceDemo, quality assurance.

Results Report of the End-to-End Test Automation Suite: SauceDemo with Playwright and TypeScript

Automated verification of web applications is a central practice in delivering quality software. This report presents, analyzes, and interprets the results obtained after executing the automation suite built as part of the technical exercise for the QA Automation Engineer position at Nuaav. The goal of the exercise was to design a maintainable suite exercising SauceDemo's functional modules by applying recognized design patterns —in particular the Page Object Model— together with test-isolation mechanisms, custom fixtures, and authentication state persistence. The following sections describe the execution methodology, the overall quantitative results, the breakdown by module and by browser, the detailed analysis of the recorded failure, an interpretive discussion, and the conclusions with their associated recommendations.

Execution Methodology

The suite was executed via the `npx playwright test` command from the repository root. The execution environment used Playwright Test as the orchestrator, with four parallel worker processes and fully parallel execution enabled (`fullyParallel: true`). The configuration defines three projects: a setup project responsible for authenticating the `standard_user` account once and persisting its session via `storageState`, and two browser projects —Chromium and Firefox— that depend on the former and reuse that session. This architecture avoids repeating the login process in every test, reducing total execution time and coupling between cases.

The timeout policy set a 45-second per-test limit, a 10-second window for assertions, and differentiated action (15 s) and navigation (20 s) budgets. Screenshots were configured to be captured on failure only (`screenshot: 'only-on-failure'`) and the debugging trace was enabled only on the first retry. Reports were generated using Playwright's built-in HTML reporter. It is worth noting that the analyzed run was performed in the local environment, where the retry policy (`retries: 1`) is disabled by design and is activated only in continuous

integration through the `CI` environment variable check; consequently, any flakiness manifests without masking.

Overall Results

The complete run comprised 37 tests and had a total duration of 4.3 minutes. Table 1 summarizes the aggregate indicators of the run. The resulting pass rate of 97.3% is obtained from the ratio of the 36 passed tests to the total of 37 executed tests.

Table 1

Aggregate indicators of the suite execution

Indicator	Value
Total tests executed	37
Tests passed	36
Tests failed	1
Flaky tests	0
Skipped tests	0
Pass rate	97.3%
Total duration	4.3 min
Worker processes	4
Browsers evaluated	2

Note. Data obtained from the Playwright HTML report for the run of June 23, 2026, 09:26. The pass rate is calculated as 36/37 × 100.

Breakdown by Test File and Functional Module

The suite is organized into six files, each aligned with a functional area of the application. Table 2 presents the distribution of cases per file, indicating the results obtained. The count reflects that most cases run across both browsers, so a single scenario contributes two tests to the total; the exceptions are the setup project and the `edge-cases.spec.ts` cases, which are counted once because they are tied to the behavior of specific accounts.

Table 2

Results by test file and functional area covered

File	Functional area	Cases	Pass	Fail
auth.setup.ts	Authentication and session persistence	1	1	0
auth.spec.ts	Login, errors, and logout	10	10	0
inventory.spec.ts	Catalogue and product sorting	10	10	0
cart.spec.ts	Shopping cart and badge counter	8	7	1
checkout.spec.ts	Checkout and validation	4	4	0
edge-cases.spec.ts	Special accounts (data and performance)	4	4	0
Total		**37**	**36**	**1**

Note. The auth.spec.ts file exercises five scenarios across two browsers (10 tests). The four edge-cases.spec.ts cases include two scenarios run in both browsers plus specific-account behaviors counted once.

Detailed Functional Coverage

Table 3 lists the specific scenarios verified for each module of the application. Coverage spans both happy paths and negative and error states, as well as edge cases associated with the atypical-behavior accounts provided by SauceDemo.

Table 3

Scenarios verified by functional module

Module	Verified scenarios	Status
Authentication	Successful login with standard_user; locked_out_user blocking; invalid credentials; empty username field; logout	Passed
Catalogue	Rendering of the six products; ascending and descending alphabetical sorting; ascending and descending price sorting	Passed
Cart	Adding one and multiple items; removal; badge update; listing on the cart page	Passed [a]
Checkout	Full three-step flow through confirmation; validation of a missing required field	Passed
Special accounts	Detection of defective images with problem_user; load within the time budget with performance_glitch_user	Passed

Note. The cart's functional coverage was fully verified in both browsers. [a] One case recorded a non-deterministic timeout in Firefox; see the failure analysis.

Comparative Performance Analysis by Browser

A notable finding of this run is the marked disparity in durations between the two browsers. Table 4 contrasts the duration of a representative set of cases executed in Chromium and Firefox. Systematically, Firefox exhibited substantially higher times —in several cases three to five times greater— which is the determining factor of the incident analyzed in the following section.

Table 4

Per-case duration comparison between Chromium and Firefox

Test case	Chromium (s)	Firefox (s)
standard_user logs in successfully	10.1	36.7
locked_out_user is blocked	9.8	40.1
invalid credentials show an error	10.0	44.2
empty username is rejected	9.9	46.1
the user can log out	6.6	24.3
adding an item updates the badge	6.0	31.0
adding multiple items increments the badge	6.0	28.8
removing an item decrements the badge	6.6	26.0
cart page lists the added items	7.4	45.7 [a]
full checkout (happy path)	8.9	23.7
checkout blocked without customer data	8.0	22.2

Note. Durations expressed in seconds. [a] Case failed due to exhausting the 45 s limit during the setup hook; it is the only incident of the run.

Failure Analysis

The only failure of the run corresponded to the *Cart › cart page lists the added items* case, executed in Firefox and located at `cart.spec.ts:33`. Table 5 summarizes its identification data. The reported error message was the exhaustion of the 45,000 ms timeout during execution of the `beforeEach` hook, and not within the test body.

Table 5

Failure identification data

Attribute	Detail
Case	Cart › cart page lists the added items
Location	cart.spec.ts:33
Browser	Firefox
Recorded duration	45.7 s (limit exhausted)
Failure point	beforeEach hook (navigation to inventory)
Message	Test timeout of 45000ms exceeded while running "beforeEach" hook

Note. Information extracted from the error context generated by Playwright for the failed test.

Root cause

The timeout occurred in the `beforeEach` hook, responsible for navigating to the inventory page and waiting for the visibility of the "Products" title before executing the test body. This circumstance is decisive for interpretation: the failure is not located in the scenario's assertions —which validate the listing of items in the cart— but in the preceding setup phase. The same case ran successfully in Chromium in 7.4 seconds, which rules out a functional defect in the application or a logic error in the test. As the comparative analysis in Table 4 shows, Firefox operated consistently slower throughout the entire suite, with times ranging from 22 to 46 seconds against Chromium's range of 4 to 12 seconds. Under the fully parallel execution configuration with four workers, one specific load instance in Firefox exceeded the 45-second budget due to system resource contention during concurrent execution. It is therefore a flakiness of environmental and performance origin, not a deterministically reproducible defect.

Mitigation built into the configuration

The suite configuration includes an automatic retry (`retries: 1`) activated exclusively in continuous integration environments through the `CI` variable check. Since the analyzed run was performed in the local environment without retries, the flakiness manifested visibly in the report. In a continuous integration environment, the retry would absorb this case and the suite

would report a fully successful result. This design decision is deliberate: keeping retries disabled locally allows flakiness to surface during development rather than remaining hidden.

Recommendations

Based on the preceding analysis, the following improvement actions are proposed:

1. Establish differentiated timeout budgets per project, assigning Firefox a higher limit than Chromium that reflects its lower observed speed in this environment.
2. Reduce resource contention by limiting the number of workers assigned to Firefox, or enable retries locally as well for performance-sensitive cases.
3. Confirm the stability of the affected case by running it in isolation with the command `npx playwright test cart.spec.ts -g "cart page lists" --project=firefox`.
4. Consider adding explicit waits on network-idle conditions in the setup hook to make initial navigation more robust against variable latencies.

Discussion

The results obtained allow us to state that the suite fulfills its purpose of verifying SauceDemo's critical flows with a high degree of reliability. Coverage spans all the required functional modules and contemplates both positive and negative scenarios, including the atypical behaviors of the special accounts. The single recorded incident illustrates a methodologically important distinction in quality assurance: the difference between a functional failure — indicative of a defect in the product— and flakiness of environmental origin, attributable to execution conditions. Correctly identifying this distinction prevents the erroneous attribution of defects to the application and directs corrective actions toward the test infrastructure. The performance disparity between browsers, documented in Table 4, does not constitute a defect in itself, but it is a risk factor that must be managed through a timeout configuration sensitive to the characteristics of each engine.

Conclusions

The execution of the automation suite achieved a pass rate of 97.3%, with 36 of 37 successful tests. Coverage verified SauceDemo's five functional modules —authentication, catalogue, cart, checkout, and logout— in the Chromium and Firefox browsers. The only failure was attributed to a non-deterministic timeout, caused by Firefox resource contention during parallel execution in the local environment, and is covered by the retry policy provided for continuous integration. Consequently, the functional quality of the application under test is considered verified across all its critical flows, and the suite constitutes a solid and maintainable foundation for integration into a continuous delivery process, subject to the performance-tuning recommendations set out in this report.